Единое задание по дисциплине «Бэкенд разработка веб-приложений»

Необходимо разработать (либо использовать уже созданное Вами ранее приложение) REST API для предметной области с поддержкой операций:

GET
POST
PUT
PATCH
DELETE

Цель работы

Спроектировать и реализовать серверное приложение, предоставляющее REST API для управления сущностями выбранной предметной области. Приложение должно корректно обрабатывать запросы клиента, возвращать HTTP-статусы, JSON-ответы и сообщения об ошибках в едином формате.

Постановка задачи

Студент должен разработать приложение, которое:

Предоставляет REST API для работы минимум с одной основной сущностью.
Поддерживает полный набор CRUD-операций.
Реализует обязательную авторизацию пользователей.
Реализует обработку ошибок в стиле REST.
Выполняет валидацию входных данных.
Возвращает ответы в формате JSON.
Использует базу данных, размещённую на сервере преподавателя. Всем, создал по отдельной бд. Версия постгреса - 17. Названия бд на скрине, приложенном с этим файлом. Для подключения: айпи - 109.198.190.115, порт - 32322, у всех юзер - student, пароль у всех - 5432.

Требования к API

Авторизация

В приложении должна быть реализована **обязательная авторизация**. Все ручки основной сущности должны быть доступны только **авторизованным пользователям**.

Минимальные требования

Студент должен реализовать:

механизм входа пользователя в систему;
проверку учетных данных;
выдачу токена или сессии после успешной авторизации;
проверку авторизации для защищённых маршрутов.

Допустимые варианты реализации

Разрешается использовать один из вариантов:

JWT-авторизация
сессионная авторизация

Предпочтительно использовать **JWT**.

Обязательные ручки авторизации

1) POST /auth/login - Вход пользователя в систему.

Требования:

- принимает логин и пароль;
- при успешной аутентификации возвращает токен доступа или информацию о сессии;
- при неверных данных возвращает ошибку авторизации.

2) POST /auth/logout - Выход из системы.

Требования:

- для сессионной авторизации завершает сессию;
- для JWT может быть реализован формально, если это предусмотрено выбранной архитектурой.

1. Основная сущность

Для каждого варианта должна быть реализована одна основная сущность. Например: книга, заказ, студент, товар, фильм и т.д.

Для сущности необходимо предусмотреть:

уникальный идентификатор id, а также не менее **5 полей**, включая:

- строковые поля
- числовые поля
- булево поле или дата/время
- обязательные и необязательные поля

Пример набора полей:

- id
- name
- description
- price
- status
- createdAt

2. Обязательные ручки

GET /entities

Получение списка всех сущностей.

Требования:

- вернуть массив объектов
- поддержать код ответа 200 OK

GET /entities/{id}

Получение одной сущности по идентификатору.

Требования:

- если объект найден — вернуть 200 OK
- если объект не найден — вернуть 404 Not Found

POST /entities

Создание новой сущности.

Требования:

- принимать JSON в теле запроса
- выполнять валидацию данных

при успешном создании вернуть:
201 Created
созданный объект
при ошибке валидации вернуть 400 Bad Request, если синтаксически некорректный запрос, или 422 Unprocessable Entity, если запрос синтаксически корректный, но не проходит бизнес-валидацию

PUT /entities/{id}

Полное обновление сущности.

Требования:

заменяет объект целиком
если объект найден и обновлён — вернуть 200 OK
если объект не найден — вернуть 404 Not Found
если тело запроса некорректно — вернуть 400 Bad Request

PATCH /entities/{id}

Частичное обновление сущности.

Требования:

обновляет только переданные поля
если объект найден и обновлён — вернуть 200 OK
если объект не найден — вернуть 404 Not Found
если переданы недопустимые поля или значения — вернуть 400 Bad Request

DELETE /entities/{id}

Удаление сущности.

Требования:

если объект найден и удалён — вернуть 200 OK или 204 No Content
если объект не найден — вернуть 404 Not Found

Обработка ошибок REST

Обязательное требование: все ошибки должны возвращаться в едином JSON-формате.

Какие ошибки должны быть реализованы обязательно

400 Bad Request

Когда:

передан некорректный JSON;
запрос имеет неверный формат;
отсутствует обязательная часть запроса, которую невозможно обработать;
передан недопустимый параметр пути или запроса.

Примеры:

повреждённый JSON в теле запроса;
id передан не в том формате;
неверный тип параметра в URL.

401 Unauthorized

Когда:

пользователь не авторизован;
отсутствует токен доступа;
передан неверный токен;
сессия недействительна.

403 Forbidden (по желанию)

Когда:

пользователь авторизован, но не имеет прав на выполнение операции.

404 Not Found

Когда:

сущность по id не найдена;
обращение к несуществующему ресурсу.

405 Method Not Allowed

Когда:

для ресурса вызван неподдерживаемый HTTP-метод.

422 Unprocessable Entity

Когда:

JSON корректный, но данные не проходят валидацию;
отсутствуют обязательные поля;
поля содержат недопустимые значения;
нарушены ограничения предметной области.

Примеры:

пустое имя;
отрицательная цена;
недопустимый статус;
дата окончания раньше даты начала;
email в неверном формате.

500 Internal Server Error

Когда:

произошла непредвиденная ошибка на сервере;
произошла ошибка подключения к базе данных;
произошёл необработанный сбой при выполнении запроса.

Валидация

Для входных данных обязательно реализовать проверки. Минимальный набор:

строковые поля не должны быть пустыми
числовые поля должны быть в допустимом диапазоне
обязательные поля должны присутствовать
значения перечислений должны быть из разрешённого списка
даты должны быть корректными

Примеры:

name — не пустая строка, длина от 2 до 100 символов
price — число больше 0
status — только active, inactive, draft

quantity — целое число не меньше 0

Что может быть в отчёте или README

В README указать:

Название проекта
Описание предметной области
Стек технологий
Как запустить проект
Описание маршрутов API
Примеры запросов и ответов
Примеры ошибок
Описание структуры проекта

Критерии оценивания

На оценку «удовлетворительно»

реализованы несколько CRUD-ручек;
реализована авторизация;
API работает;

На оценку «хорошо»

+есть базовая обработка 401, 404, 422.
+есть корректная валидация;
+единый формат ошибок;
+защищены все маршруты от неавторизованного пользователя;

На оценку «отлично»

+есть README.
+обработка 400, 401, 404, 405, 422, 500;

Что должно быть в отчёте

1) В отчёте должна быть описана ваша предметная область.
2) Для каждой ручки должны быть прикреплены скриншоты непосредственно их в коде и скриншоты их применения через сваггер или любым другим методом. Результат применения я должен проверить у себя в бд.
3) Продемонстрировать, что неавторизованный пользователь не может пользоваться ручками.
4) Предоставить скриншотами то, как у вас реализована валидация в коде.
5) Обработку ошибок тоже приложить скриншотами.
6) Если есть readme и/или env – приложить к отчёту.

Варианты

Для тех, кто с нуля будет делать

Вариант 1. Книги

Сущность: **Book**
Поля:

- `id`
- `title`
- `author`
- `genre`
- `year`
- `available`

Вариант 2. Студенты

Сущность: **Student**
Поля:

- `id`
- `fullName`
- `groupName`
- `email`
- `age`
- `active`

Вариант 3. Курсы

Сущность: **Course**
Поля:

- `id`
- `title`
- `description`
- `durationHours`
- `teacher`
- `published`

Вариант 4. Фильмы

Сущность: **Movie**
Поля:

- `id`
- `title`
- `director`
- `releaseYear`
- `rating`
- `available`

Вариант 5. Товары интернет-магазина

Сущность: **Product**
Поля:

- `id`
- `name`
- `category`
- `price`
- `stockQuantity`
- `active`

Вариант 6. Заказы

Сущность: **Order**
Поля:

- `id`
- `customerName`
- `productName`
- `quantity`
- `totalPrice`
- `status`

Вариант 7. Клиенты

Сущность: **Client**
Поля:

- `id`
- `fullName`
- `phone`
- `email`
- `discount`
- `verified`

Вариант 8. Преподаватели

Сущность: **Teacher**
Поля:

- `id`
- `fullName`
- `department`
- `email`
- `experienceYears`
- `active`

Вариант 9. Задачи

Сущность: **Task**
Поля:

- `id`
- `title`
- `description`
- `priority`
- `deadline`
- `completed`

Вариант 10. События

Сущность: **Event**
Поля:

- `id`
- `title`
- `location`
- `date`
- `maxParticipants`
- `open`

Вариант 11. Рецепты

Сущность: **Recipe**
Поля:

- `id`
- `name`
- `cuisine`
- `cookingTime`
- `difficulty`
- `published`

Вариант 12. Автомобили

Сущность: **Car**
Поля:

- `id`
- `brand`
- `model`
- `year`
- `price`
- `available`

Вариант 13. Отели

Сущность: **Hotel**
Поля:

- `id`
- `name`
- `city`
- `stars`
- `roomsCount`
- `open`

Вариант 14. Билеты

Сущность: **Ticket**
Поля:

- `id`
- `eventName`
- `holderName`
- `seatNumber`
- `price`
- `used`

Вариант 15. Подписки

Сущность: **Subscription**
Поля:

- `id`
- `serviceName`
- `userName`
- `startDate`
- `endDate`
- `active`